
Trabajo Práctico Integrador

Programación I

Gestión de Datos de Países en Python:
filtros, ordenamientos y estadísticas

Institución	UTN · Tecnatura Universitaria en Programación
Materia	Programación I
Integrantes	Juan Retamar Francisco Goette
Fecha	Junio 2026

Índice

1. Marco Teórico	3
1.1 Listas	3
1.2 Diccionarios	3
1.3 Funciones	3
1.4 Estructuras Condicionales	4
1.5 Ordenamientos	4
1.6 Estadísticas Básicas	4
1.7 Archivos CSV	4
2. Decisiones Técnicas y Arquitectura	5
2.1 Estructura modular del proyecto	5
2.2 Diagrama de flujo	5
2.3 Capturas de pantalla	6
3. Dificultades y Conclusiones	9
4. Bibliografía / Webgrafía	10
5. Links del proyecto	10

1. Marco Teórico

1.1 Listas

Una lista en Python es una colección ordenada y mutable de elementos, capaz de almacenar objetos de cualquier tipo. Se definen con corchetes `[]` y permiten acceder a sus elementos por índice (base 0), agregar con `append()`, recorrer con bucles `for` y modificar cualquier elemento directamente.

En este proyecto, la lista *países* es la estructura central que persiste en memoria durante toda la sesión. Cada elemento es un diccionario que representa un país. La lista se carga desde el CSV al inicio y se pasa como argumento a todas las funciones que la necesitan.

```
países = [] # lista vacía países.append({'nombre': 'Argentina', ...}) # agregar for p in países: print(p['nombre']) # recorrer
```

1.2 Diccionarios

Un diccionario (`dict`) es una colección de pares clave-valor, donde cada clave es única e inmutable. Se define con llaves `{}` y se accede a los valores mediante sus claves. Son la estructura ideal para representar entidades con múltiples atributos.

Cada país se modela como un diccionario con cuatro claves: nombre, población, superficie y continente. Esto permite acceder a cualquier dato del país por nombre semántico en lugar de índice numérico, haciendo el código más legible y mantenible.

```
pais = { "nombre": "Brasil", "poblacion": 213993437, "superficie": 8515767, "continente": "América" } print(pais["nombre"]) # Brasil
```

1.3 Funciones

Una función es un bloque de código reutilizable que agrupa una tarea específica. Se define con la palabra clave **def**, puede recibir parámetros y devolver valores con **return**. La modularización — principio 'una función = una responsabilidad' — mejora la legibilidad, facilita las pruebas y reduce la duplicación de código.

El proyecto aplica modularización estricta: cada archivo contiene funciones con una sola responsabilidad. Por ejemplo, `leer_paises()` solo lee; `guardar_paises()` solo escribe; `buscar_por_nombre()` solo busca. Las funciones reciben la lista *países* como parámetro y no dependen de variables globales.

```
def buscar_por_nombre(países): nombre = input('Nombre: ').strip() resultados = [p for p in países if nombre.lower() in p['nombre'].lower()] for p in resultados: print(p['nombre'])
```

1.4 Estructuras Condicionales

Las estructuras condicionales (`if / elif / else`) permiten ejecutar distintos bloques de código según el valor de una expresión booleana. Son fundamentales para el control de flujo y la validación de datos.

Se utilizan extensamente en el menú principal (ramificar según la opción elegida), en las validaciones de entrada (detectar campos vacíos, tipos incorrectos, duplicados) y en los filtros (comparar rangos de población o superficie). El manejo de errores con `try/except` complementa las condicionales para capturar excepciones en conversiones de tipo.

```
opcion = input("Elegí: ").strip() if opcion == "1": agregar_pais(países) elif opcion == "2": actualizar_pais(países) else: print("[ERROR] Opción inválida.")
```

1.5 Ordenamientos

Python provee la función built-in `sorted()` que retorna una nueva lista ordenada sin modificar la original. Acepta los parámetros **key** (función que extrae el criterio de comparación) y **reverse** (True para orden descendente). El uso de lambda como función anónima hace la expresión compacta.

El módulo `ordenamiento.py` permite ordenar los países por nombre (string), población o superficie (enteros), en cualquier dirección. El algoritmo subyacente de `sorted()` es Timsort, con complejidad $O(n \log n)$ en el peor caso.

```
paises_ord = sorted( paises, key=lambda p: p['poblacion'], reverse=True ) for p in paises_ord: print(p['nombre'], p['poblacion'])
```

1.6 Estadísticas Básicas

Python incluye funciones built-in para cálculos estadísticos elementales: `max()` y `min()` con parámetro `key` para encontrar el elemento extremo según un criterio; `sum()` para sumar un iterable; y `len()` para contar elementos. Combinadas con expresiones generadoras, permiten calcular promedios con una sola línea.

El módulo estadisticas.py calcula el país con mayor y menor población, los promedios de población y superficie, y la distribución de países por continente usando un diccionario acumulador.

```
mayor = max(países, key=lambda p: p['poblacion']) menor = min(países, key=lambda p:
p['poblacion']) prom = sum(p['poblacion'] for p in países) / len(países) print(f'Mayor:
{mayor["nombre"]} - Prom: {prom:,.0f}')
```

1.7 Archivos CSV

Un archivo CSV (Comma-Separated Values) almacena datos tabulares como texto plano, con campos separados por comas y un registro por línea. Python incluye el módulo estándar csv con las clases DictReader (lee filas como diccionarios usando la cabecera como claves) y DictWriter (escribe diccionarios con cabecera automática).

archivos.py centraliza toda la E/S: leer_paises() abre el CSV, convierte tipos (int para población y superficie) y retorna la lista; guardar_paises() sobrescribe el archivo completo tras cada modificación. El uso de encoding='utf-8' garantiza el soporte de caracteres especiales como tildes y ñ.

```
import csv with open("paises.csv", newline="", encoding="utf-8") as f: lector =
csv.DictReader(f) for fila in lector: pais = { "nombre": fila["nombre"].strip(),
"poblacion": int(fila["poblacion"]), }
```

2. Decisiones Técnicas y Arquitectura

2.1 Estructura Modular del Proyecto

Se eligió una arquitectura de módulos independientes donde cada archivo .py tiene una única responsabilidad. Esto facilita la lectura, las pruebas individuales y la división de tareas entre los integrantes del equipo. La lista países se pasa como argumento entre funciones, evitando el uso de variables globales y manteniendo el estado centralizado en main.py.

Archivo	Responsabilidad	Funciones principales
main.py	Menú y coordinación	main(), mostrar_menu(), menu_filtros()
archivos.py	Persistencia CSV	leer_paises(), guardar_paises()
gestor.py	Alta y modificación	agregar_pais(), actualizar_pais()
busqueda.py	Consultas y filtros	buscar_por_nombre(), filtrar_por_*
ordenamiento.py	Ordenamiento	ordenar_paises()
estadisticas.py	Cálculo de indicadores	mostrar_estadisticas()
países.csv	Dataset persistente	24 países · 4 campos

2.2 Diagrama de Flujo — Operaciones Principales

El siguiente diagrama muestra el flujo de ejecución del sistema: desde la carga del CSV hasta la selección de cada opción del menú y el retorno al ciclo principal.

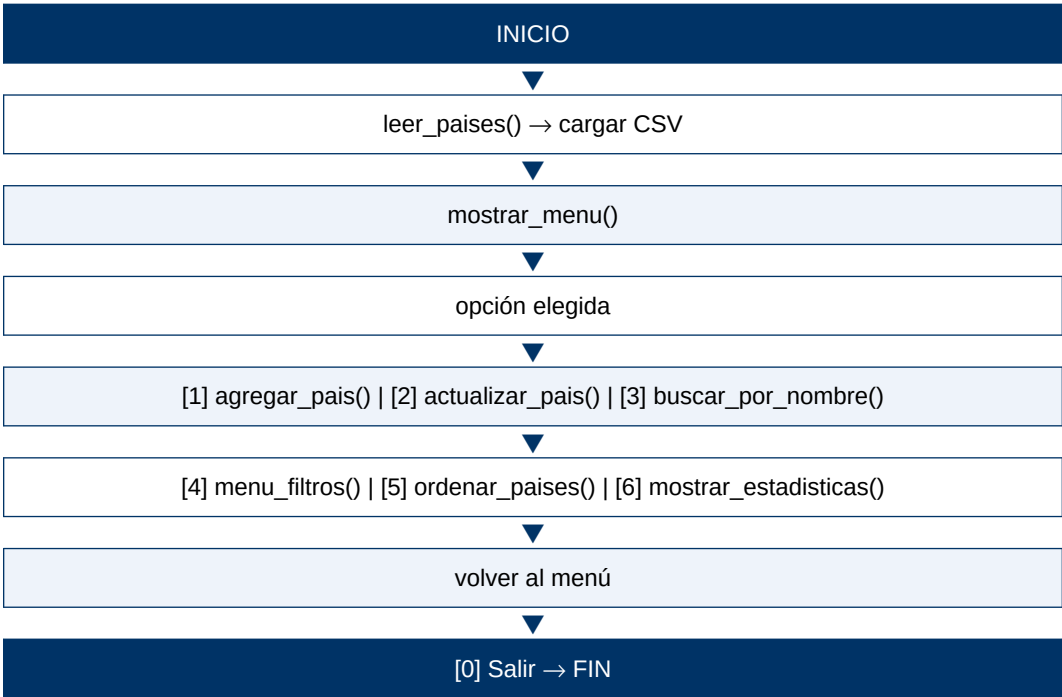


Figura 1 — Diagrama de flujo del sistema de Gestión de Países.

2.3 Capturas de Pantalla — Ejecución del Sistema

Las siguientes capturas muestran la ejecución del programa en consola para cada una de las funcionalidades implementadas.

```
franc/Desktop/TPI-programacion-/main.py
Elegí una opción: 6

--- ESTADÍSTICAS ---

País con mayor población: China (1,412,600,000 hab.)
País con menor población: Curazao (192,000 hab.)

Promedio de población: 185,472,896 hab.
Promedio de superficie: 2,024,072 km²

Cantidad de países por continente:
- América: 6 país/es
- Europa: 6 país/es
- Asia: 6 país/es
- África: 4 país/es
- Oceanía: 2 país/es
- America del Sur: 1 país/es
```

Captura 1 — Menú principal del sistema

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINA
PS C:\Users\franc\Desktop\TPI-programacion-> & C
franc/Desktop/TPI-programacion-/main.py
GESTIÓN DE PAÍSES - UTN
=====
1. Agregar país
2. Actualizar país
3. Buscar país por nombre
4. Filtrar países
5. Ordenar países
6. Ver estadísticas
0. Salir
=====
Elegí una opción: 1

--- AGREGAR PAÍS ---
Nombre del país: Curazao
Población: 192000
Superficie en km²: 444
Continente: America del Sur
[OK] Datos guardados correctamente.
[OK] País 'Curazao' agregado correctamente.
```

Captura 2 — Agregar país (caso éxito y validación)

```

=====
Elegí una opción: 2

--- ACTUALIZAR PAÍS ---
Nombre del país a actualizar: Argentina
Nueva población (actual: 456894): 456700
Nueva superficie (actual: 4357332): 4357332
[OK] Datos guardados correctamente.
[OK] País 'Argentina' actualizado correctamente.

```

Captura 3 — Actualizar datos de un país

```

Elegí una opción: 3

--- BUSCAR PAÍS POR NOMBRE ---
Ingresá el nombre o parte del nombre: Colombia

Se encontraron 1 resultado/s:
- Colombia | Población: 51,874,024 | Superficie: 1,141,748 km² | Continente: América

=====
      GESTIÓN DE PAÍSES - UTN
=====

1. Agregar país
2. Actualizar país
3. Buscar país por nombre
4. Filtrar países
5. Ordenar países
6. Ver estadísticas
0. Salir

```

Captura 4 — Búsqueda por nombre (coincidencia parcial)

```

franc/Desktop/TPI-programacion-/main.py

Elegí una opción (1-3): 1

--- FILTRAR POR CONTINENTE ---
Ingresá el continente: America del Sur

Países en America del Sur:
- Curazao | Población: 192,000 | Superficie: 444 km²

=====
      GESTIÓN DE PAÍSES - UTN
=====

1. Agregar país
2. Actualizar país
3. Buscar país por nombre
4. Filtrar países
5. Ordenar países
6. Ver estadísticas
0. Salir

```

Captura 5 — Filtros: por continente

```
Elegí una opción: 5

--- ORDENAR PAÍSES ---
¿Por qué campo querés ordenar?
1. Nombre
2. Población
3. Superficie

Elegí una opción (1-3): 1

¿En qué orden?
1. Ascendente
2. Descendente

Elegí una opción (1-2): 1

Países ordenados por nombre (ascendente):
- Alemania | Población: 83,149,300 | Superficie: 357,022 km² | Continente: Europa
- Arabia Saudita | Población: 34,813,871 | Superficie: 2,149,690 km² | Continente: Asia
- Argentina | Población: 456,700 | Superficie: 4,357,332 km² | Continente: América
- Australia | Población: 25,499,884 | Superficie: 7,692,024 km² | Continente: Oceanía
- Brasil | Población: 213,993,437 | Superficie: 8,515,767 km² | Continente: América
- Chile | Población: 19,458,310 | Superficie: 756,102 km² | Continente: América
- China | Población: 1,412,600,000 | Superficie: 9,596,960 km² | Continente: Asia
- Colombia | Población: 51,874,024 | Superficie: 1,141,748 km² | Continente: América
- Curazao | Población: 192,000 | Superficie: 444 km² | Continente: America del Sur
- Egipto | Población: 102,334,404 | Superficie: 1,001,449 km² | Continente: África
- España | Población: 47,351,567 | Superficie: 505,990 km² | Continente: Europa
- Etiopía | Población: 114,963,588 | Superficie: 1,104,300 km² | Continente: África
- Francia | Población: 67,391,582 | Superficie: 551,695 km² | Continente: Europa
```

Captura 6 — Ordenamiento por nombre ascendente

3. Dificultades y Conclusiones

Obstáculos técnicos encontrados

- **Conversión de tipos en el CSV:** Al leer el archivo con DictReader, todos los valores llegan como cadenas de texto. Fue necesario convertir explícitamente poblacion y superficie a int, y agregar manejo de errores con try/except ValueError para detectar filas malformadas sin interrumpir la ejecución.
- **Validación de entradas:** Garantizar que el usuario no ingresara campos vacíos, caracteres inválidos o nombres duplicados requirió múltiples condicionales. Se centralizó la validación de texto en la función auxiliar es_texto_valido() para no repetir la misma lógica en agregar_pais() y actualizar_pais().
- **Codificación de caracteres:** Los países con tildes y caracteres especiales (Japón, Perú, España, etc.) generaban errores en Windows si no se especificaba encoding='utf-8' tanto en la lectura como en la escritura del CSV.
- **Búsqueda case-insensitive:** Lograr que la búsqueda por nombre y los filtros por continente no distinguieran mayúsculas de minúsculas requirió aplicar .lower() tanto a la entrada del usuario como a los valores almacenados.

Reflexiones finales

Este TPI nos permitió consolidar los conceptos fundamentales de Python en un proyecto real con persistencia de datos. La arquitectura modular facilitó enormemente la división de tareas: cada integrante pudo trabajar sobre módulos independientes sin generar conflictos en el código.

El mayor aprendizaje fue comprender cómo las estructuras de datos básicas — listas de diccionarios — pueden ser suficientes para representar un dataset funcional sin necesidad de bases de datos externas. La combinación de sorted() con lambda y el módulo csv demostró el poder de la biblioteca estándar de Python para tareas cotidianas de procesamiento de información.

También aprendimos la importancia de validar siempre las entradas del usuario: un dato incorrecto en el CSV puede propagarse silenciosamente si no se controla en el punto de lectura. El manejo explícito de excepciones y los mensajes descriptivos de error/éxito mejoran significativamente la experiencia de uso.

4. Bibliografía / Webgrafía

- [1] Python Software Foundation. (2024). Python 3 Documentation — Built-in Types: list, dict. <https://docs.python.org/3/library/stdtypes.html>
- [2] Python Software Foundation. (2024). csv — CSV File Reading and Writing. <https://docs.python.org/3/library/csv.html>
- [3] Python Software Foundation. (2024). Sorting HOW TO. <https://docs.python.org/3/howto/sorting.html>
- [4] Python Software Foundation. (2024). Built-in Functions: sorted, max, min, sum, len. <https://docs.python.org/3/library/functions.html>
- [5] Python Software Foundation. (2024). Errors and Exceptions. <https://docs.python.org/3/tutorial/errors.html>
- [6] Real Python. (2023). Reading and Writing CSV Files in Python. <https://realpython.com/python-csv/>
- [7] Real Python. (2023). How to Use Python Lambda Functions. <https://realpython.com/python-lambda/>

5. Links del Proyecto

Recurso	Enlace
Repositorio GitHub	https://github.com/juan22retamar/TPI-programacion-
Video demostrativo	https://youtu.be/9VRfuF5Ym1M
Documentación PDF	Incluida en el repositorio raíz